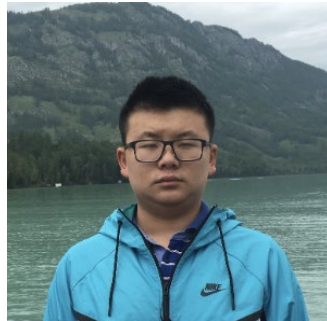


Adversarial Robustness on Insertion-Deletion Streams



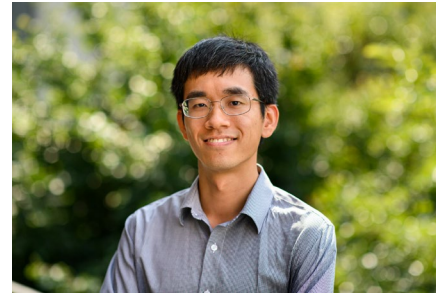
Elena Gribelyuk
Princeton



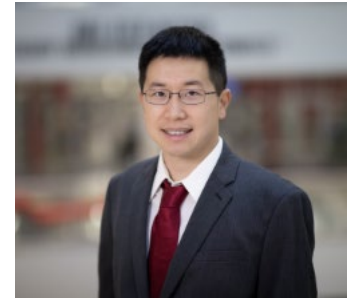
Honghao Lin
CMU



David P. Woodruff
CMU



Huacheng Yu
Princeton



Samson Zhou
Texas A&M

Model #1: Streaming Model

- **Input:** Elements of an underlying data set S , which arrives sequentially
- **Output:** Evaluation (or approximation) of a given function



Internet traffic



Sensor networks



Stock markets

Lots of problems...

- **Graph problems:** Cut sparsification
- **Geometric problems:** Clustering, facility location
- **Statistical problems:** Heavy-hitters, norm/moment estimation, quantile estimation
- **Algebraic problems:** Subspace embeddings, regression, low-rank approximation
- **String problems:** pattern matching, periodicity
- **Others:** CSPs, coding theory, submodular optimization, etc

Frequency Vector

- Given a set S of m elements from $[n]$, let f_i be the frequency of element i . (How often it appears)

$$1\ 1\ 2\ 1\ 2\ 1\ 1\ 2\ 3 \rightarrow [5, 3, 1, 0] := f$$

- **Insertion-only stream**: each update only increases coordinates
- **Insertion-deletion stream**: when updates can increase or decrease coordinates
- **Goal**: Use space *sublinear* in the stream length m and dimension n

Frequency Moments

- Given a set S of m elements from $[n]$, let x_i be the frequency of element i . (How often it appears)
- Let F_p be the frequency moment of the vector:

$$F_p = |x_1|^p + |x_2|^p + \dots + |x_n|^p$$

- **Goal:** Given a set S of m elements from $[n]$ and an accuracy parameter ε , output a $(1 + \varepsilon)$ -approximation to F_p
- **Motivation:** Entropy estimation, linear regression, subroutine for many other problems

F_2 Estimation

- AMS algorithm gives $(1 + \varepsilon)$ -approximation to F_2 using $O\left(\frac{1}{\varepsilon^2} \log n\right)$ bits of communication [AlonMatiasSzegedy99]
- Let $s \in \{-1, +1\}^n$ be a random sign vector
- Let $Z = \langle s, f \rangle = s_1 f_1 + s_2 f_2 + \cdots + s_n f_n$
- $Z^2 = \sum_{i,j \in [n]} s_i s_j f_i f_j$
- $\sum_{i=j} s_i s_j f_i f_j = (f_1^2 + f_2^2 + \cdots + f_n^2) := F_2$

AMS Algorithm

- $E[Z^2] = \sum_{i,j \in [n]} s_i s_j f_i f_j$
- $\text{Var}[Z^2] \leq E[Z^4] = \sum_{a,b,c,d \in [n]} s_a s_b s_c s_d f_a f_b f_c f_d \leq 6F_2^2$
- **Chebyshev's inequality**: the average of $O\left(\frac{1}{\varepsilon^2}\right)$ independent instances gets $(1 + \varepsilon)$ -approximation with probability at least $\frac{2}{3}$
- AMS algorithm gives $(1 + \varepsilon)$ -approximation to F_2 using $O\left(\frac{1}{\varepsilon^2} \log n\right)$ bits of communication [AlonMatiasSzegedy99]

Model #2: Adversarially Robust Streaming

- **Input:** Updates to an underlying vector x , which arrive sequentially and *adversarially*
- **Output:** Evaluation (or approximation) of a given function
- **Goal:** Use space *sublinear* in the size m of the input S

[Ben-EliezerJayaramWoodruffYogev18]

- **Adversarial robustness:** “Future queries may depend on previous responses”
- **Motivation:** Database queries, optimization

Adversarially Robust Streaming

- **Input:** Updates to an underlying vector x , which arrive sequentially and *adversarially*
- **Output:** Evaluation (or approximation) of a given function
- **Goal:** Use space *sublinear* in the size m of the input S



Attacker

➤ Output # of distinct elements
(number of nonzero coordinates)



Algorithm

Adversarially Robust Streaming

- **Input:** Updates to an underlying vector x , which arrive sequentially and *adversarially*
- **Output:** Evaluation (or approximation) of a given function
- **Goal:** Use space *sublinear* in the size m of the input S



Attacker

$$x_1 \leftarrow x_1 + 1$$

1



Algorithm

Adversarially Robust Streaming

- **Input:** Updates to an underlying vector x , which arrive sequentially and *adversarially*
- **Output:** Evaluation (or approximation) of a given function
- **Goal:** Use space *sublinear* in the size m of the input S



Attacker

$$\begin{aligned}x_1 &\leftarrow x_1 + 1 \\x_2 &\leftarrow x_2 + 1\end{aligned}$$

2



Algorithm

Adversarially Robust Streaming

- **Input:** Updates to an underlying vector x , which arrive sequentially and *adversarially*
- **Output:** Evaluation (or approximation) of a given function
- **Goal:** Use space *sublinear* in the size m of the input S



Attacker

$$\begin{aligned}x_1 &\leftarrow x_1 + 1 \\x_2 &\leftarrow x_2 + 1 \\x_3 &\leftarrow x_3 + 1\end{aligned}$$

3



Algorithm

Adversarially Robust Streaming

- **Input:** Updates to an underlying vector x , which arrive sequentially and *adversarially*
- **Output:** Evaluation (or approximation) of a given function
- **Goal:** Use space *sublinear* in the size m of the input S



Attacker

$$\begin{aligned}x_1 &\leftarrow x_1 + 1 \\x_2 &\leftarrow x_2 + 1 \\x_3 &\leftarrow x_3 + 1 \\x_1 &\leftarrow x_1 + 1\end{aligned}$$

4



Algorithm

Adversarially Robust Streaming

- **Input:** Updates to an underlying vector x , which arrive sequentially and *adversarially*
- **Output:** Evaluation (or approximation) of a given function
- **Goal:** Use space *sublinear* in the size m of the input S



Attacker

$$\begin{aligned}x_1 &\leftarrow x_1 + 1 \\x_2 &\leftarrow x_2 + 1 \\x_3 &\leftarrow x_3 + 1 \\x_1 &\leftarrow x_1 + 1\end{aligned}$$

4



Algorithm

“Attack” on AMS

- Can learn whether $s_i = s_j$ from $\langle s, e_i + e_j \rangle$
 - Let $f_i = 1$ if $s_i = s_1$ and $f_i = -1$ if $s_i \neq s_1$
 - $Z = \langle s, f \rangle = s_1 f_1 + \cdots + s_n f_n = m$ and $Z^2 = m^2$ deterministically
-
- What happened? Randomness of algorithm not independent of input

Insertion-Only Streams

- Space $\tilde{O}\left(\frac{1}{\varepsilon^2} \log n\right)$ algorithm for $(1 + \varepsilon)$ -approximation to F_2 moment [AlonMatiasSzegedy99, BlasiokDingNelson17]
- Space $\tilde{O}\left(\frac{1}{\varepsilon^2} \log n\right)$ **robust** algorithm for $(1 + \varepsilon)$ -approximation to F_2 moment [WoodruffZhou21]

Bounded Computation Paths

- Consider a deterministic adversary, who will update their strategy based on the algorithm's output sequence

[Ben-EliezerJayaramWoodruffYogev20]

- If there are only k possible algorithmic *output* sequences, there are only k possible adversarial *input* sequences, then we can union bound over all k possible sequences, i.e., set failure probability to $\frac{1}{k \cdot \text{poly}(m)}$
- If space dependency for non-robust algorithm is $\log \frac{1}{\delta}$, this is $O(\log km)$ space overhead for robust algorithm

Bounded Computation Paths

- **Summary:** If output changes at most T times, there are $k \approx m^{O(T)}$ possible streams
- If space dependency for non-robust algorithm is $\log \frac{1}{\delta}$, this is $O(T \log m)$ space overhead for robust algorithm
- For $(1 + \varepsilon)$ -approximation for polynomially-bounded functions on insertion-only streams, $T = O\left(\frac{1}{\varepsilon} \log m\right)$

Insertion-Deletion Streams

- Space $\tilde{O}\left(\frac{1}{\varepsilon^2} \log^2 n\right)$ algorithm for $(1 + \varepsilon)$ -approximation to F_2 moment [AlonMatiasSzegedy99]
- Space for **robust** streaming algorithm for approximation to F_2 moment?
 - Sketch switching / bounded computation paths: $\tilde{O}(m)$ space
 - If $m > n$, maintain vector f deterministically with $O(n \log n)$ bits.
 - Using differential privacy: $\tilde{O}(\sqrt{m})$ space

Linear Sketch

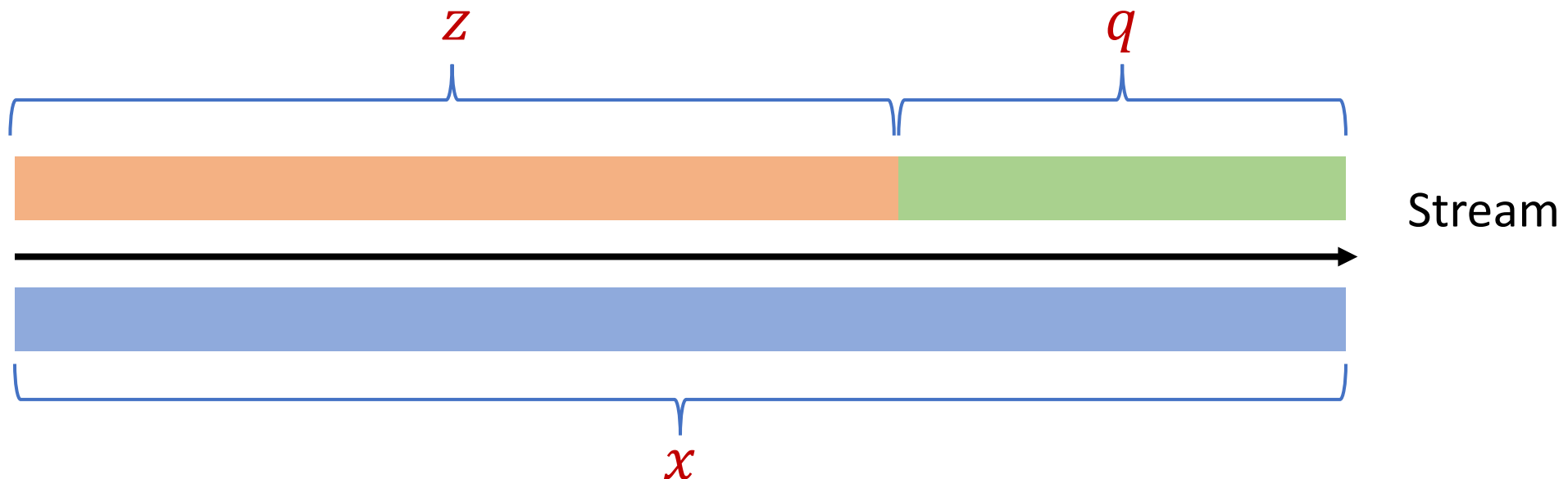
- Algorithm maintains Ax for some matrix A throughout the stream
- In **classical** streaming: as well be linear sketch [JiangLiuYu26]
 - Adaptive attack approximately learns the rowspan of the sketching matrix A .
 - Hard distribution: query $x \in \text{Ker}(A)$ or $x = 0^n$.
- Any multiplicative approximation algorithm for F_p estimation based on a linear sketch over an **adaptively** chosen stream requires $\Omega(n)$ space [HardtWoodruff13, GribelyukLinWoodruffYuZhou25]
- Conjecture: $\Omega(n)$ space is necessary for any adversarially robust insertion-deletion algorithm

Our Results, Adversarial Robustness on Insertion-Deletion Streams

- We refute the $\Omega(n)$ conjecture!! We use a non-linear sketch!!
- Space $\text{poly}\left(\frac{1}{\varepsilon}, \log n\right)$ algorithm for $(1 + \varepsilon)$ -approximation to F_2 moment
- Space $\text{poly}\left(\frac{1}{\varepsilon}, \log n\right)$ algorithm for L_2 heavy-hitters
- Space $n^{1/c}$ overhead for algorithm for $2^{O(c)}$ -approximation to functions with approximate triangle inequality; trade-off between space blow-up and approximation factor

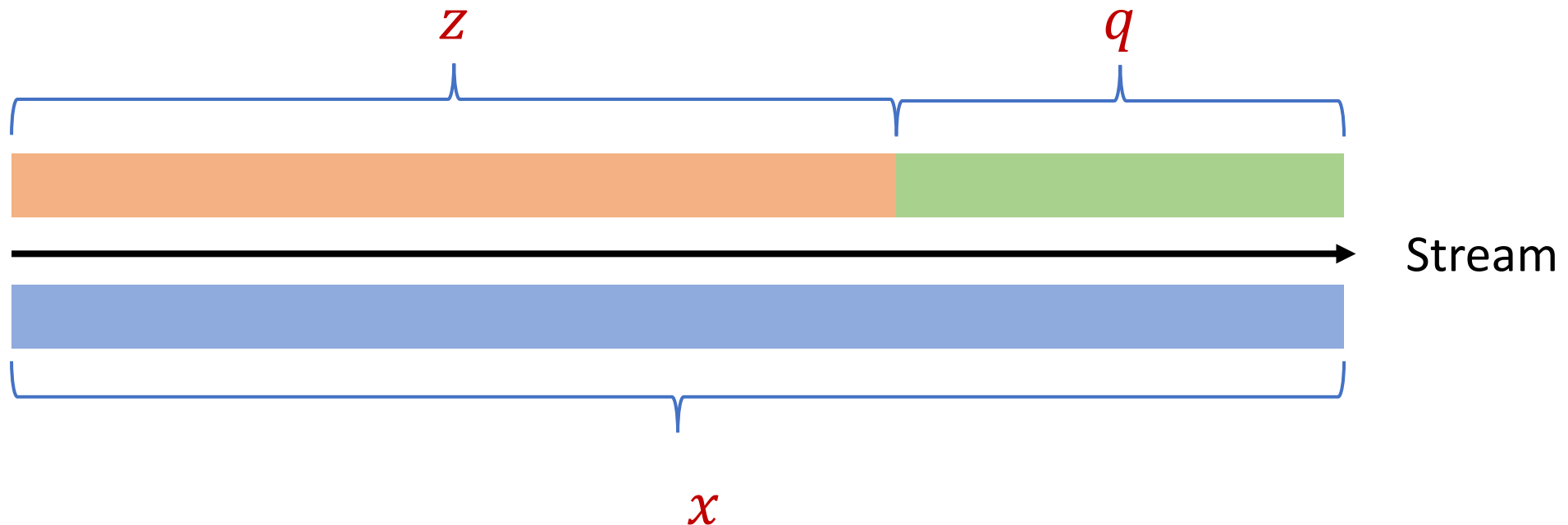
Intuition

- Decompose frequency vector x as $x = z + q$
- Use sketch matrix Az for z and sketch matrix Bq for q (independent F_2 sketches)
- Maintain Ax in the background
- Focus on protecting A for now



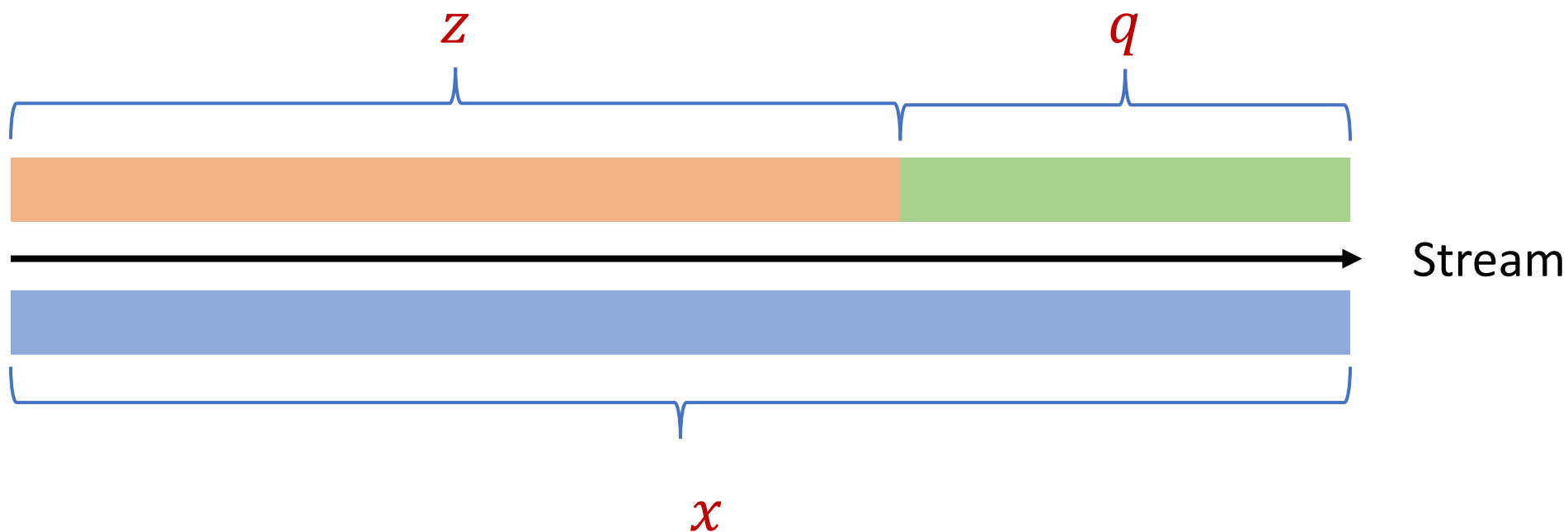
Intuition

- Since $x = z + q$, then $Bx = Bz + Bq$
- Initialized and maintained Bq at the beginning of the block
- Don't have Bz , so how to estimate $\|x\|_2^2$?



Intuition

- Since $x = z + q$, then $\|x\|_2^2 = \|z + q\|_2^2 = \|z\|_2^2 + 2\langle z, q \rangle + \|q\|_2^2$



Intuition

- Since $x = z + q$, then $\|x\|_2^2 = \|z + q\|_2^2 = \|z\|_2^2 + 2\langle z, q \rangle + \|q\|_2^2$
- Then either:
 - $\langle z, q \rangle$ is small, so that $\|x\|_2^2 \approx \|z\|_2^2 + \|q\|_2^2 \approx \|Az\|_2^2 + \|Bq\|_2^2$
 - $\langle z, q \rangle$ is large, so that q is “well-aligned” with z , and we

Learn information about z from q

Estimator, Corrector, Learner

- Three ingredients:
 - An *estimator*, whose role is to output estimates of $\|z + q\|_2^2$
 - For example, output $\|z\|_2^2 + \|q\|_2^2$, but actually more complex
 - A *learner*, whose role is to learn z . When the estimator is incorrect, the learner will gain information about z from q
 - A *corrector*, whose role is to inform the learner/estimator when the estimate is incorrect

Estimator, Corrector, Learner

- Learner maintains a vector z' , which is a linear combination of the queries q on which the algorithm was incorrect
- Estimator outputs $\|z - z'\|_2^2 + \|z' + q\|_2^2$ as estimate for $\|z + q\|_2^2$, where the first term is estimated using $Az - Az'$, and the second term is estimated using $Bz' + Bq$
- Corrector uses Ax to detect when estimator is incorrect

Estimator, Corrector, Learner

- **Sanity check #1:** For $z' = 0$, then $\|z - z'\|_2^2 + \|z' + q\|_2^2$ corresponds to earlier estimator $\|z\|_2^2 + \|q\|_2^2$
- **Sanity check #2:** For $z' = z$, then $\|z - z'\|_2^2 + \|z' + q\|_2^2$ corresponds to correct value $\|z + q\|_2^2$
- How many interactions with corrector before $z' \approx z$?

Update Rule

- Estimator is $\|z - z'\|_2^2 + \|z' + q\|_2^2$
- Incorrect approximation by estimator implies

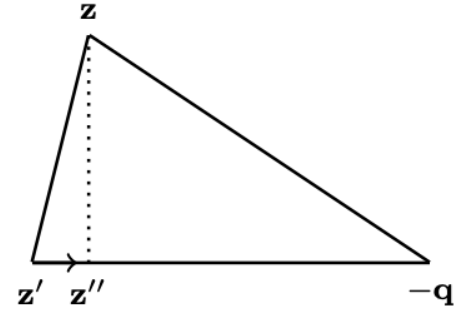
$$\left| \underbrace{\|z - z'\|_2^2 + \|z' + q\|_2^2}_{\text{Estimator}} - \underbrace{\|z + q\|_2^2}_{\text{Answer}} \right| \geq \underbrace{\varepsilon \cdot \|z + q\|_2^2}_{\text{Error margin}}$$

and ultimately,

$$|\langle z - z', z' + q \rangle| \geq \frac{\varepsilon}{2} \cdot \|z + q\|_2^2$$

- Assume WLOG $\underbrace{\langle z - z', z' + q \rangle}_{\text{Learner error}} \geq \frac{\varepsilon}{2} \cdot \|z + q\|_2^2$ (other case similar)

Update Rule



- On query q where estimator errs, learner uses update rule $z' \leftarrow z' + \alpha(q + z')$ and truncates small coordinates
- Let $z'' = z' + \alpha(q + z')$ be the new iterate after updating
- Will show: $\underbrace{\|z - z''\|_2^2}_{\text{New learner error}} \leq (1 - \varepsilon^2) \cdot \underbrace{\|z - z'\|_2^2}_{\text{Old learner error}}$
- Coordinates bounded by $\text{poly}(n) \rightarrow$ at most $O\left(\frac{1}{\varepsilon^2} \log n\right)$ updates

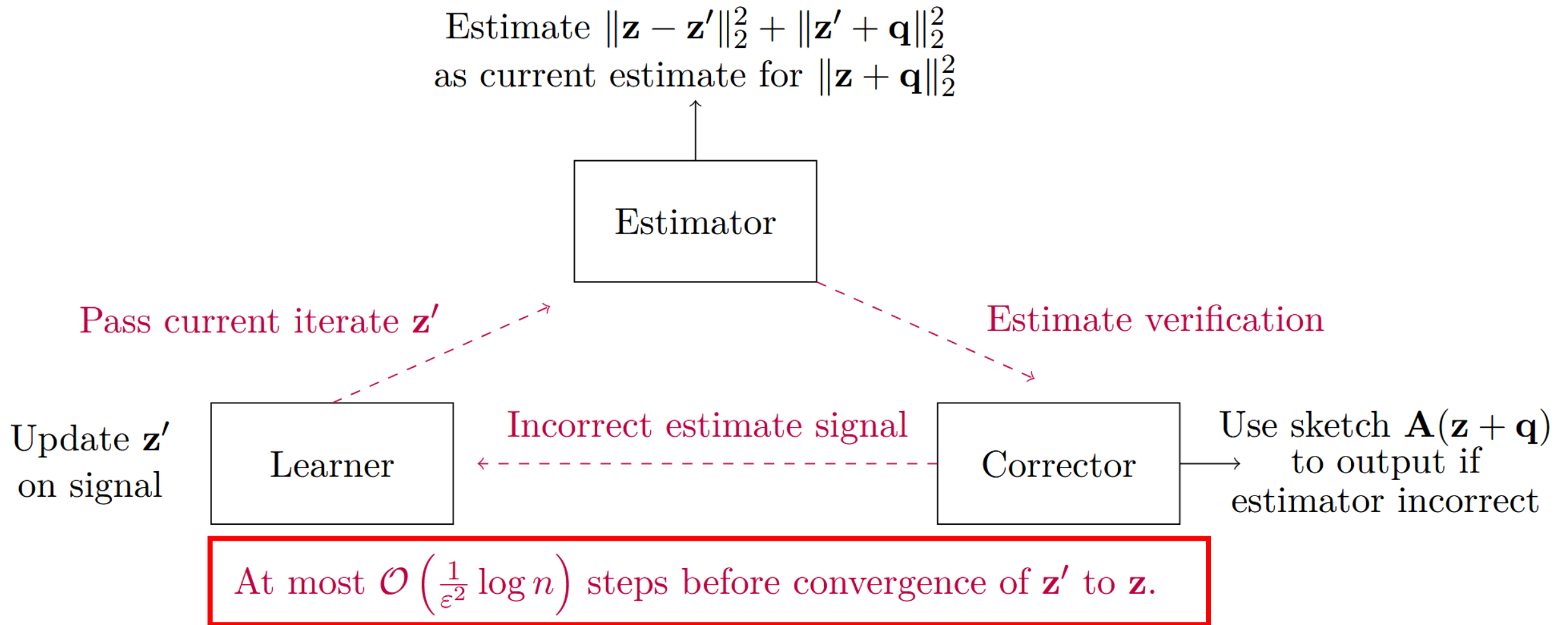
Progress

$$\underbrace{\|z - z''\|_2^2}_{\text{New learner error}} = \|z - z' - \alpha(q + z')\|_2^2$$
$$= \underbrace{\|z - z'\|_2^2}_{\text{Old learner error}} + \alpha^2 \cdot \|q + z'\|_2^2 - 2\alpha \cdot \langle z - z', q + z' \rangle$$

- **Recall**: due to incorrect estimator, $\langle z - z', z' + q \rangle \geq \frac{\varepsilon}{2} \cdot \|z + q\|_2^2$

- By setting $\alpha \approx \varepsilon \cdot \frac{\|z - z'\|_2}{\|q + z'\|_2}$ (can estimate this), we make progress:

$$\|z - z''\|_2^2 \leq (1 - \varepsilon^2) \cdot \|z - z'\|_2^2$$

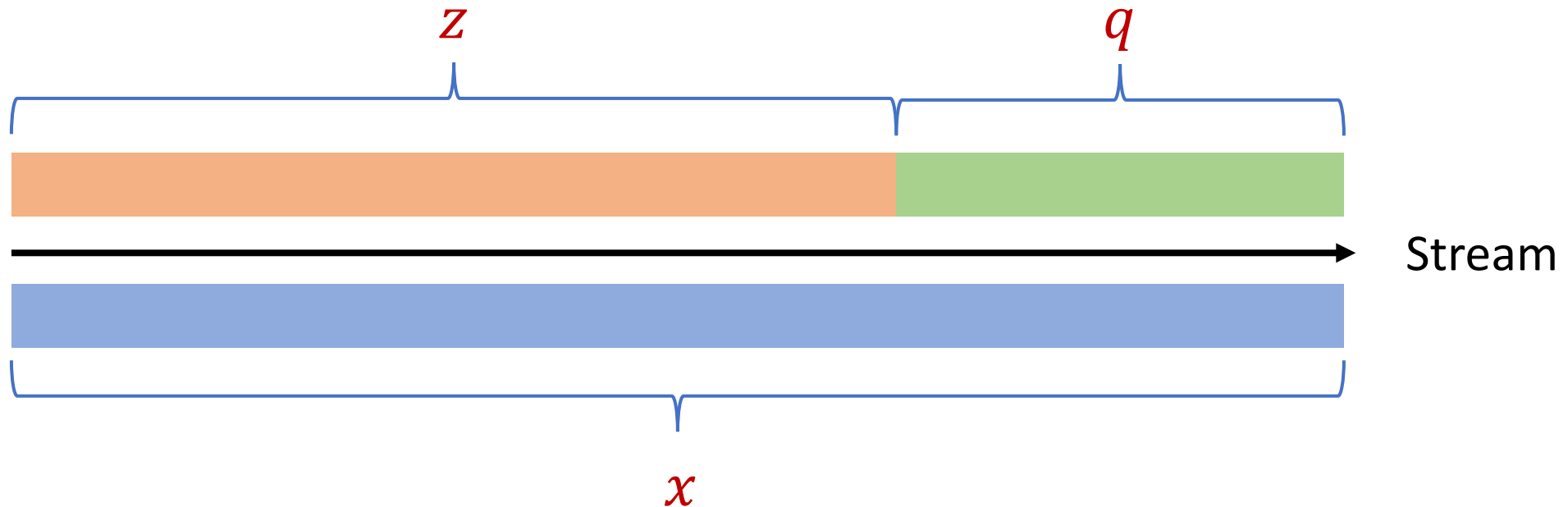


The estimator computes a current estimate, the corrector verifies its accuracy, and the learner updates its internal state $\mathbf{z}'$ based on incorrect estimates

* Output $\|A(\mathbf{z} + \mathbf{q})\|_2^2$ when estimator is incorrect.

Progress

- **Key**: Can learn z using $O\left(\frac{1}{\varepsilon^2} \log n\right)$ adaptive queries to corrector
- Still need A to be robust over z and B to be robust over q
- Recurse on each block!



Recursion

- Let $q = q_1 + q_2$, where q_2 is still evolving over the stream
- Estimator is $\|z - z'\|_2^2 + \|z' + q\|_2^2$ and suppose iterate z' is *fixed*
- Then:
 - First term is fixed
 - Second term is $\|z' + q_1 + q_2\|_2^2$ and estimator must be robust

Recursion

- Robust estimate for $\|z' + q_1 + q_2\|_2^2$
- Recurse using estimator-corrector-learner framework:
 - Corrector for $\|z' + q_1 + q_2\|_2^2$
 - Learner has an iterate u to learn $z' + q_1$
 - Estimator uses $\|z' + q_1 - u\|_2^2 + \|u + q_2\|_2^2$ as estimate

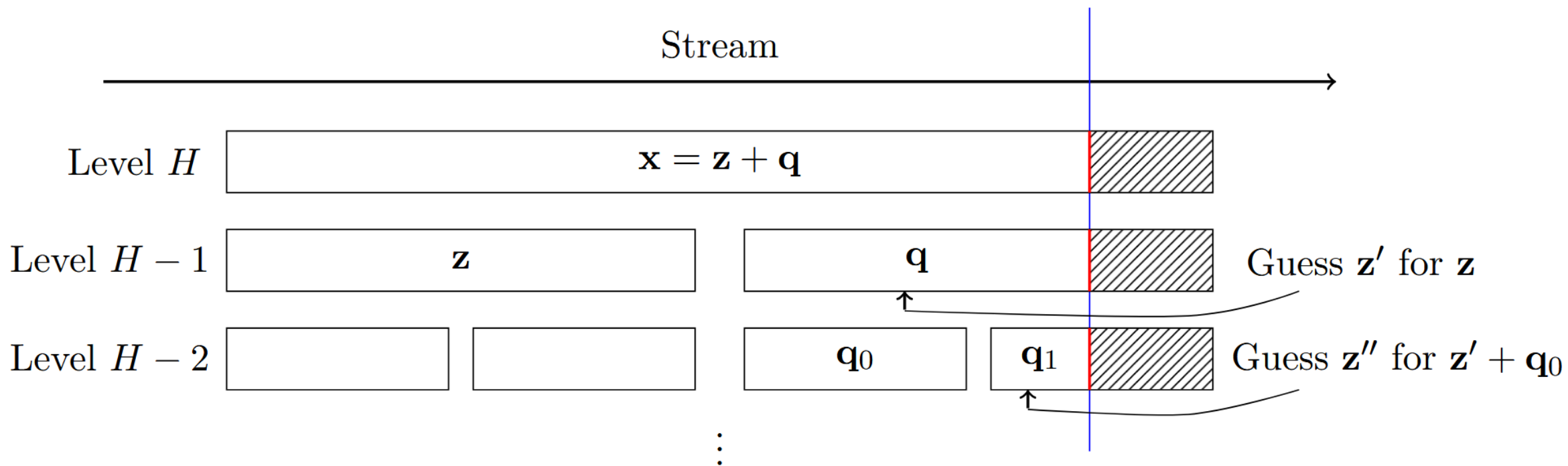


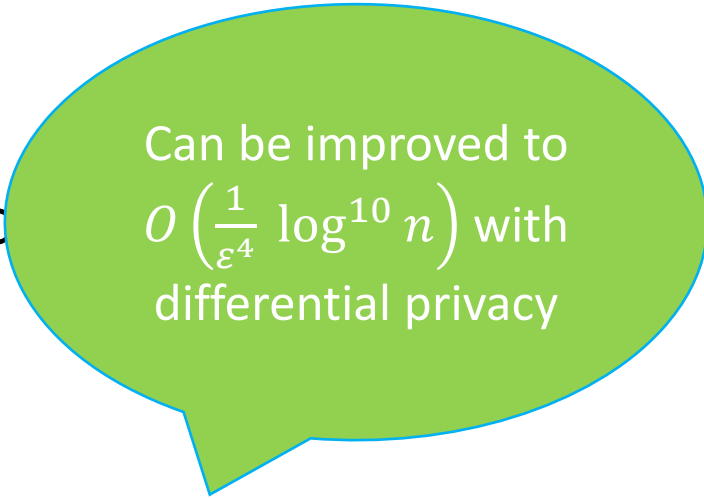
Fig. 3: Example of recursive tree structure on stream with $B = 2$ blocks and H levels. Level $H - 1$ outputs $\|\mathbf{z} - \mathbf{z}'\|_2^2 + \|\mathbf{z}' + \mathbf{q}\|_2^2$ as estimate for $\|\mathbf{x}\|_2^2 = \|\mathbf{z} + \mathbf{q}\|_2^2$. Level $H - 2$ outputs $\|\mathbf{z}' + \mathbf{q}_0 - \mathbf{z}''\|_2^2 + \|\mathbf{z}'' + \mathbf{q}_1\|_2^2$ as estimate for $\|\mathbf{z}' + \mathbf{q}\|_2^2 = \|\mathbf{z}' + \mathbf{q}_0 + \mathbf{q}_1\|_2^2$.

Recursion Tree

- Suppose each node in recursion tree of height H has B child blocks
- Must have $B \geq \frac{1}{\epsilon^2} \log n$, since adversary can force you to change z' at most $O\left(\frac{1}{\epsilon^2} \log n\right)$ times
- For $B^H \geq m$, suffices to set $H = O(\log m)$

Space Complexity

- Each corrector must be robust to $B = O\left(\frac{1}{\varepsilon^2} \log n\right)$ adaptive queries in each of the B blocks in the next level.
- Each AMS algorithm uses space $O\left(\frac{1}{\varepsilon^2} \log^2 n\right)$ and should maintain B sketches at each of the $O(\log m)$ levels!
- Rescale ε to $\frac{\varepsilon}{\log m}$ as multiplicative error compounds
- Total space $O\left(\frac{1}{\varepsilon^6} \log^{11} n\right)$ for $\log m = O(\log n)$



Can be improved to $O\left(\frac{1}{\varepsilon^4} \log^{10} n\right)$ with differential privacy

Future Directions

- Improve the concrete $\text{poly}\left(\frac{1}{\varepsilon}, \log n\right)$ factors
- Get $(1 + \varepsilon)$ -approximation for other functions, e.g., F_p moment estimation for $p > 2$, etc., with $\text{poly}\left(\frac{1}{\varepsilon}, \log n\right)$ overhead

A word cloud featuring the phrase "Thank you" in numerous languages. The central focus is the English words "thankyou" in large, bold, blue lowercase letters. Surrounding it are various translations in different colors and sizes:

- Spanish:** gracias (red), gracias (green)
- Italian:** grazie (light blue)
- German:** danke (orange), danken (blue)
- French:** merci (dark blue)
- Japanese:** arigato (pink), arigatou (purple), kiasai (yellow)
- Chinese:** 谢谢 (black)
- Hindi:** धन्यवाद (brown)
- Russian:** спасибо (grey)
- Portuguese:** obrigado (maroon)
- Turkish:** teşekkür ederim (green)
- Arabic:** شكرا (yellow)
- Hebrew:** תודה (black)
- Swedish:** tack (pink)
- Dutch:** dank je (blue)
- Czech:** děkuji (blue)
- Slovak:** ďakujem (blue)
- Polsish:** dziękuję (blue)
- Ukrainian:** дякую (blue)
- Bulgarian:** благодаря (blue)
- Serbian/Croatian/Bosnian:** hvala (red), hvala (blue)
- Malay:** terima kasih (black)
- Indonesian:** terima kasih (black)
- Vietnamese:** cảm ơn (black)
- Korean:** 감사합니다 (black)
- Mongolian:** рхамт (black)
- Yiddish:** דאנק (black)
- Tagalog:** salamat (black)
- Thai:** ขอบคุณ (black)
- Lao:** ຄຳຂອບໃຈ (black)
- Khmer:** គោរពអំណរ (black)
- Siamese:** ขอขอบคุณ (black)
- Chamorro:** agasado (black)
- Filipino:** magandang aral (black)
- Marathi:** धन्यवाद (black)
- Gujarati:** ધન્યવાદ (black)
- Urdu:** شکریہ (black)
- Punjabi:** ਧੰਨਵਾਦ (black)
- Assamese:** ধন্যবাদ (black)
- Odia:** ଧନ୍ୟବାଦ (black)
- Bengali:** ধন্যবাদ (black)
- Tamil:** தயவுசெய்து (black)
- Malayalam:** നന്ദി (black)
- Kannada:** ಧನ್ಯವಾದ (black)
- Telugu:** ధన్యవాదం (black)
- Marathi:** धन्यवाद (black)
- Gujarati:** ધન્યવાદ (black)
- Urdu:** شکریہ (black)
- Punjabi:** ਧੰਨਵਾਦ (black)
- Assamese:** ধন্যবাদ (black)
- Odia:** ଧନ୍ୟବାଦ (black)
- Bengali:** ধন্যবাদ (black)
- Tamil:** தயவுசெய்து (black)
- Malayalam:** നന്ദി (black)
- Kannada:** ಧನ್ಯವಾದ (black)
- Telugu:** ధన్యవాదం (black)

- **Graph problems:** Cut sparsification
- **Geometric problems:** Clustering, facility location, MST
- **Statistical problems:** Heavy-hitters, norm/moment estimation, quantile estimation
- **Algebraic problems:** Subspace embeddings, regression, low-rank approximation

Heavy-Hitters

- Let $\frac{\varepsilon}{2} \cdot e_i$ be a small scaling of the elementary vector
- If x_i is a **heavy-hitter**, i.e., $|x_i|^2 \geq \varepsilon \cdot \|x\|_2^2$ is a heavy-hitter, then increasing x by $\frac{\varepsilon}{2} \cdot \|x\|_2 \cdot e_i$, will increase $\|x\|_2^2$ by a “**lot**”
- If x_i is **not a heavy-hitter**, i.e., $|x_i|^2 < \frac{\varepsilon}{2} \cdot \|x\|_2^2$ is a heavy-hitter, then increasing x by $\frac{\varepsilon}{2} \cdot \|x\|_2 \cdot e_i$, will increase $\|x\|_2^2$ by a “**little**”

Heavy-Hitters

- These two cases differ by a constant factor, and thus distinguishable by our robust F_2 estimation algorithm
- Test all coordinates $i \in [n]$
- Space $\text{poly}\left(\frac{1}{\varepsilon}, \log n\right)$ algorithm for L_2 heavy-hitters

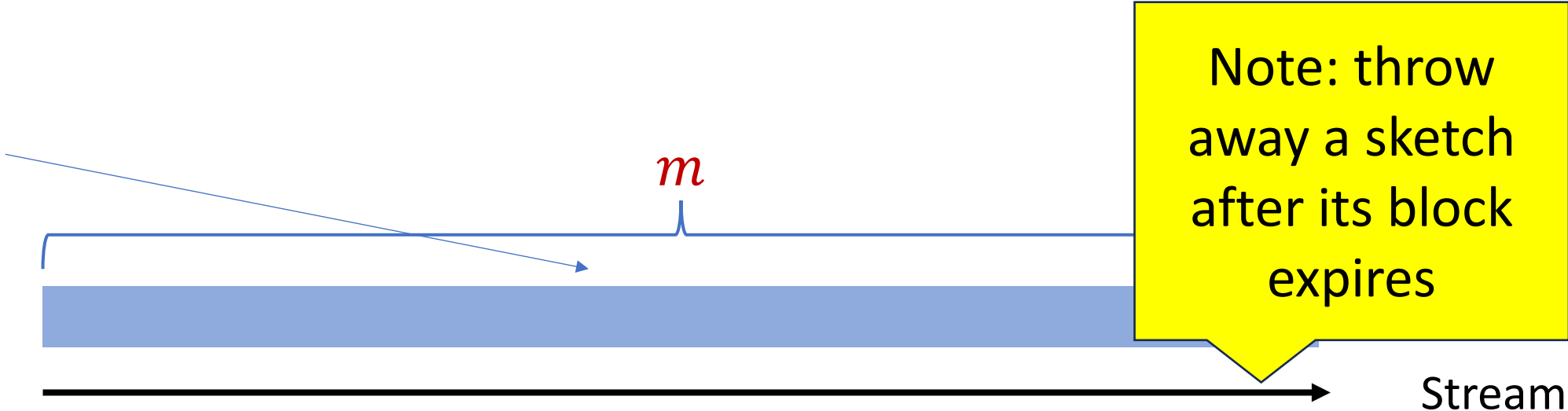
Triangle Inequality

- Corrector flags when $F(z - z') + F(z' + q) \geq 4 \cdot F(z + q)$
 - $F(z - z') + F(z' + q) \leq 2F(z - z') + F(z + q)$
 - $F(z + q) \leq \frac{2}{4}F(z - z') + \frac{1}{4}F(z + q)$
 - $F(z + q) \leq \frac{2}{3}F(z - z')$
- Set new z' to be $-q$
- Generalizes to approximate triangle inequality

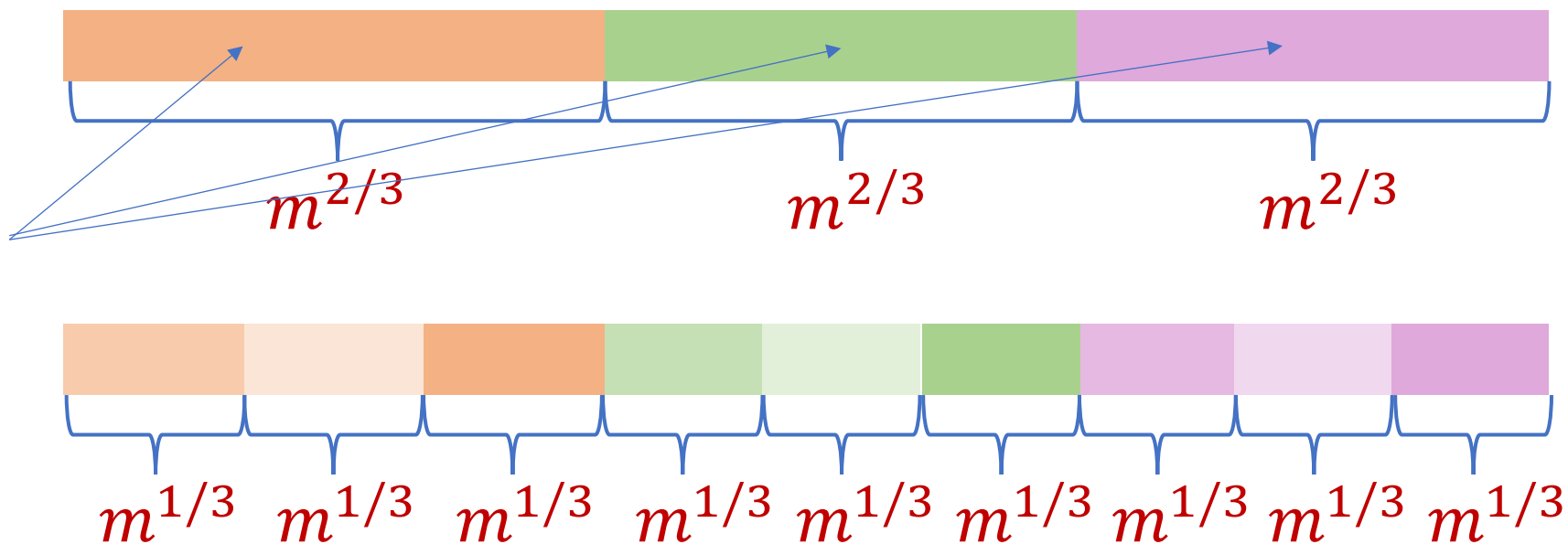
Triangle Inequality

- Constant-factor compounding across each level, so set height of recursion tree to be a parameter C
- Each block has size $n^{1/C}$
- Algorithm must be robust to $n^{1/C}$ adaptive queries
- $n^{O(1/C)}$ space and $2^{O(C)}$ -approximation

One F_2 -sketch at top level correct on $m^{1/3} \varepsilon^{-2} \log m$ adaptive queries



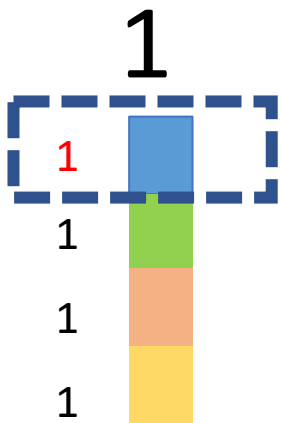
Independent F_2 -sketch for each block of middle level, correct on $m^{1/3} \varepsilon^{-2} \log m$ adaptive queries



Independent F_2 -sketch for each block of $m^{1/3}$ adaptive queries at bottom level

Sketch Switching

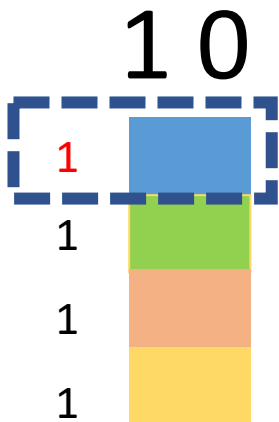
- Initialize k copies of a streaming algorithm at the beginning
- Use an instance of the algorithm but freeze the current output
- Each time that the next instance has value more than $(1 + O(\varepsilon))$ times the frozen output, we *switch* to the next instance and freeze its output



- Example: estimate the number of ones in the stream (2-approx.)

Sketch Switching

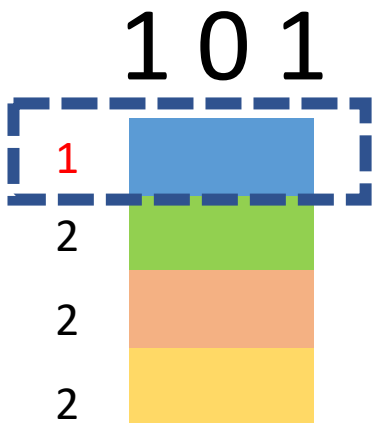
- Initialize k copies of a streaming algorithm at the beginning
- Use an instance of the algorithm but freeze the current output
- Each time that the next instance has value more than $(1 + O(\varepsilon))$ times the frozen output, we *switch* to the next instance and freeze its output



- Example: estimate the number of ones in the stream (2-approx.)

Sketch Switching

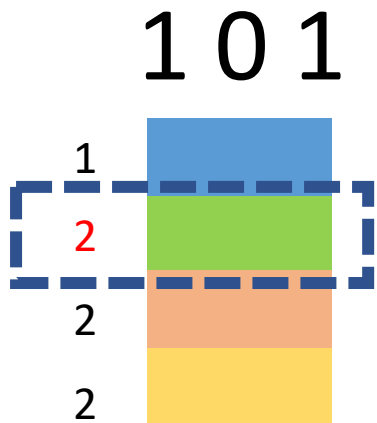
- Initialize k copies of a streaming algorithm at the beginning
- Use an instance of the algorithm but freeze the current output
- Each time that the next instance has value more than $(1 + O(\varepsilon))$ times the frozen output, we *switch* to the next instance and freeze its output



- Example: estimate the number of ones in the stream (2-approx.)

Sketch Switching

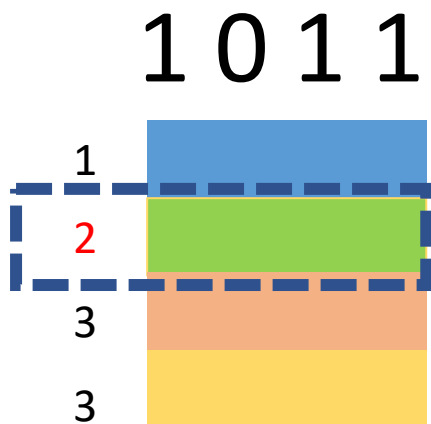
- Initialize k copies of a streaming algorithm at the beginning
- Use an instance of the algorithm but freeze the current output
- Each time that the next instance has value more than $(1 + O(\varepsilon))$ times the frozen output, we *switch* to the next instance and freeze its output



- Example: estimate the number of ones in the stream (2-approx.)

Sketch Switching

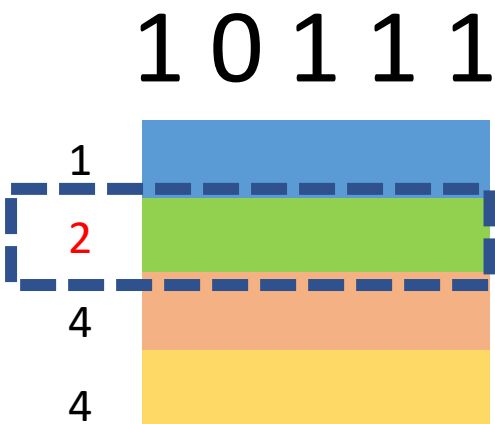
- Initialize k copies of a streaming algorithm at the beginning
- Use an instance of the algorithm but freeze the current output
- Each time that the next instance has value more than $(1 + O(\varepsilon))$ times the frozen output, we *switch* to the next instance and freeze its output



- Example: estimate the number of ones in the stream (2-approx.)

Sketch Switching

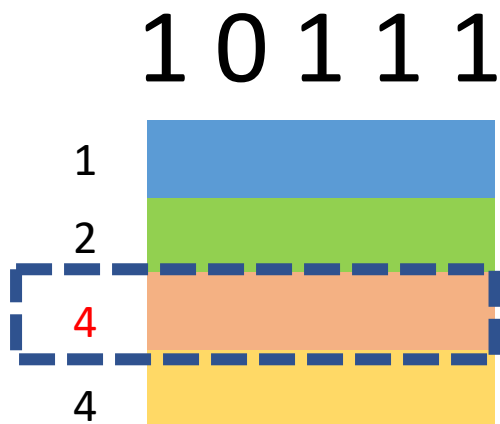
- Initialize k copies of a streaming algorithm at the beginning
- Use an instance of the algorithm but freeze the current output
- Each time that the next instance has value more than $(1 + O(\varepsilon))$ times the frozen output, we *switch* to the next instance and freeze its output



- Example: estimate the number of ones in the stream (2-approx.)

Sketch Switching

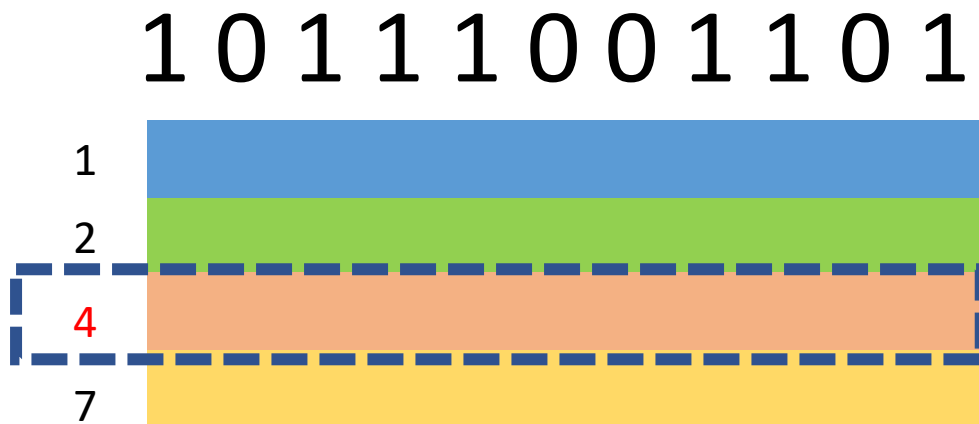
- Initialize k copies of a streaming algorithm at the beginning
- Use an instance of the algorithm but freeze the current output
- Each time that the next instance has value more than $(1 + O(\varepsilon))$ times the frozen output, we *switch* to the next instance and freeze its output



- Example: estimate the number of ones in the stream (2-approx.)

Sketch Switching

- Initialize k copies of a streaming algorithm at the beginning
- Use an instance of the algorithm but freeze the current output
- Each time that the next instance has value more than $(1 + O(\varepsilon))$ times the frozen output, we *switch* to the next instance and freeze its output



- Example: estimate the number of ones in the stream (2-approx.)
- Number of ones stream is at least 4 and at most 8
- 4 is a good approximation